

Actividad 1.

La plataforma de código abierto Docker, permite la creación, implementación y administración de aplicaciones utilizando la tecnología de contenedores.

Investiga los componentes de Docker que se muestran en la figura 2.

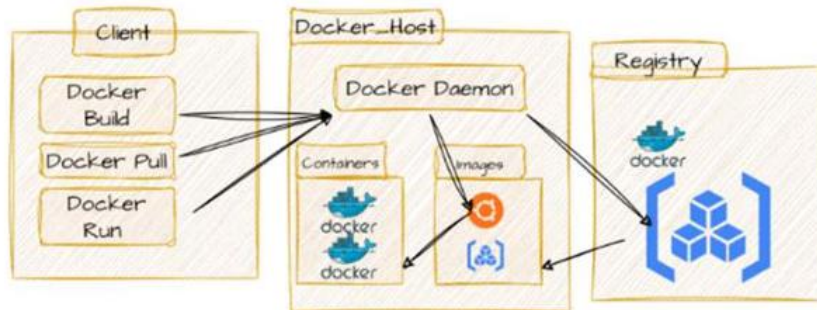


Figura 2

Los componentes de Docker que se muestran en la figura anterior son:

- **Client**

Es la interfaz mediante la cual el usuario interactúa con Docker, normalmente a través de la línea de comandos.

- **docker build:** Crea una imagen a partir de un Dockerfile.
- **docker pull:** Descarga una imagen desde un registro (como Docker Hub).
- **docker run:** Crea y ejecuta un contenedor a partir de una imagen.

- **Docker Host**

Es la máquina donde Docker está instalado. Aquí se ejecutan los contenedores.

- **Docker Daemon (dockerd):** Es el proceso principal que ejecuta Docker. Recibe instrucciones del cliente, administra imágenes, contenedores, redes y volúmenes.
- **Images:** Son plantillas inmutables que contienen todo lo necesario para ejecutar una aplicación (código, librerías, variables de entorno, etc.).

- **Containers:** Son instancias en ejecución de las imágenes. Son ligeros, portables y aislados.

- Registry

Es el repositorio donde se almacenan las **imágenes Docker**.

- El más conocido es **Docker Hub**, pero pueden haber registros privados o de terceros.
- Las imágenes se **suben (push)** o se **descargan (pull)** desde registry.

Actividad 2.

Investigar como instalar Docker en la distribución de Linux de su preferencia. Indicar el proceso de instalación y capturas de pantalla. Una vez instalado ejecutar `Docker --version`

En mi caso, instalaré Docker en mi terminal de Ubuntu para Windows. Primero que nada, se actualizan los paquetes del sistema, esto para que no haya ningún problema.

```
kylos@DELL7200:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
Get another security update through Ubuntu Pro with 'esm-apps' enabled:
 7zip
Learn more about Ubuntu Pro at https://ubuntu.com/pro
The following packages will be upgraded:
  apparmor apport apport-core-dump-handler cloud-init distro-info-data dmsetup gir1.2-packagekitglib-1.0
  gtk-update-icon-cache landscape-client landscape-common libapparmor1 libcryptsetup12 libdevmapper1.02.1
  libgtk-3-0t64 libgtk-3-bin libgtk-3-common libnetplan1 libnss-systemd libpackagekit-glib2-18 libpam-systemd
  libsystemd-shared libsystemd0 libudev1 libxml2 linux-libc-dev netplan-generator netplan.io openssh-client
  packagekit packagekit-tools pci.ids python3-apport python3-netplan python3-problem-report
  python3-software-properties rsyslog snapd software-properties-common systemd systemd-dev systemd-resolved
  systemd-sysv systemd-timesyncd ubuntu-pro-client ubuntu-pro-client-l10n udev
46 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
3 standard LTS security updates
Need to get 50.3 MB/51.2 MB of archives.
After this operation, 1229 kB of additional disk space will be used.
Do you want to continue? [Y/n]
```

```
apport-autoreport.service is a disabled or a static unit not running, not starting it.
Processing triggers for hicolor-icon-theme (0.17-2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libgl1:amd64 (2.80.0-6ubuntu3.2) ...
Processing triggers for dbus (1.14.10-4ubuntu4.1) ...
Setting up libgtk-3-0t64:amd64 (3.24.41-4ubuntu1.3) ...
Setting up libgtk-3-bin (3.24.41-4ubuntu1.3) ...
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
kylos@DELL7200:~$ █
```

Después, se instalan las dependencias necesarias para que, al momento de instalar Docker, no haya algún problema.

```
kylos@DELL7200:~$ sudo apt install apt-transport-https ca-certificates curl software-properties-common
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
curl is already the newest version (8.5.0-2ubuntu10.6).
curl set to manually installed.
software-properties-common is already the newest version (0.99.49.2).
software-properties-common set to manually installed.
The following NEW packages will be installed:
  apt-transport-https
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 3974 B of archives.
After this operation, 35.8 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 apt-transport-https all 2.7.14build2 [3974 B]
Fetched 3974 B in 1s (7153 B/s)
Selecting previously unselected package apt-transport-https.
(Reading database ... 52498 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_2.7.14build2_all.deb ...
Unpacking apt-transport-https (2.7.14build2) ...
Setting up apt-transport-https (2.7.14build2) ...
kylos@DELL7200:~$ █
```

Luego, se agrega la clave GPG oficial de Docker.

```
kylos@DELL7200:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
kylos@DELL7200:~$ █
```

Seguido de ello, se agrega el repositorio de Docker, esto mediante un “echo” y un “tee” (comandos de Linux)

```
kylos@DELL7200:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
kylos@DELL7200:~$ █
```

Después, se procede a instalar Docker Engine.

```
kylos@DELL7200:~$ sudo apt install docker-ce
```

Y, por último, se procede a validar que Docker se haya instalado correctamente, esto con el comando "Docker --version".

```
kylos@DELL7200:~$ docker --version  
Docker version 28.1.1, build 4eba377
```

Donde queda comprobado que Docker ha sido instalado correctamente.

Actividad 3

En Docker, una imagen es una plantilla ligera y ejecutable que incluye todo lo necesario para ejecutar una aplicación y un contenedor es una imagen arrancada y que ejecuta una única aplicación.

En esta sección del laboratorio se crea un contenedor Docker que imprima por pantalla Hola.

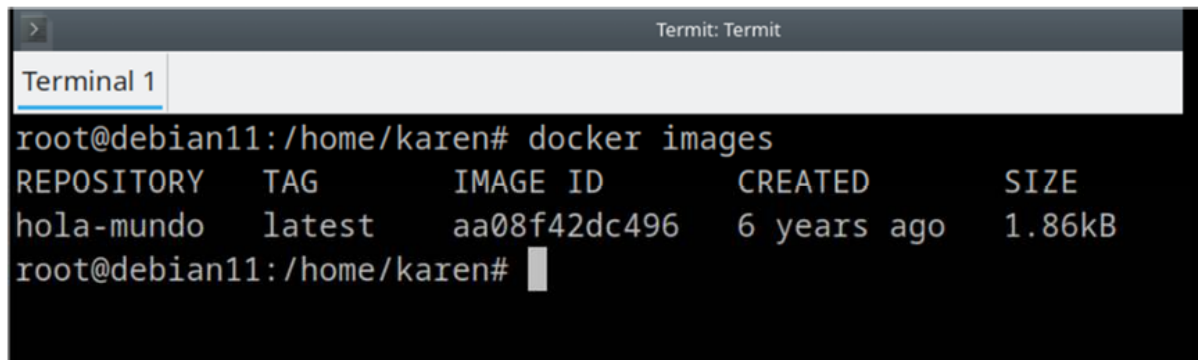
Para ello crear y ejecutar el primer contenedor Docker que simplemente imprima por pantalla *hola mundo*.

```
$docker run hola-mundo
```

Como podrás ver, el comando anterior simplemente **imprime un mensaje por pantalla**, que incluye, entre otras cosas, la sentencia Hola de Docker! y termina, devolviendo el control de la consola al usuario.

Ahora se revisan las imágenes y contenedores con los que se cuentan hasta ahora.

Usando ***\$docker images*** se visualiza la imagen que se descargó (al no estar presente en nuestro docker)



```
Termit: Termit
Terminal 1
root@debian11:/home/karen# docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hola-mundo    latest    aa08f42dc496   6 years ago    1.86kB
root@debian11:/home/karen#
```

Con ***\$docker ps*** o ***\$docker container ls*** con vemos los contenedores activos en ejecución

Con ***\$docker container ls -a*** se visualizan tanto los que se encuentran en ejecución como los detenidos

Ejecutar ambos y decir si se tienen contenedores en ejecución. _____

Donde creé y ejecuté el primer contenedor Docker de la siguiente manera:

```
kylos@DELL7200:~$ docker run hola-mundo
Unable to find image 'hola-mundo:latest' locally
latest: Pulling from library/hola-mundo
708e56201c93: Pull complete
Digest: sha256:7fd85c550d60b54cf950722d75b0307ebb374329833b87b03d8a0eb59efdb487
Status: Downloaded newer image for hola-mundo:latest

¡Hola de DockerCon EU 2015 (Barcelona)!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hola-mundo" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

Además de mostrarse la imagen de la siguiente manera (ya después de haberse descargado)

```
kylos@DELL7200:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
hola-mundo    latest    aa08f42dc496   6 years ago    1.86kB
kylos@DELL7200:~$
```

Ahora, con el comando “docker ps”, veo los contenedores activos en ejecución.

```
kylos@DELL7200:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
kylos@DELL7200:~$
```

Y que como se observa, no hay ningún ejecutándose. Ahora, usando “docker container ls -a”, podré ver los dockers que estén en ejecución, así como los detenidos, viéndose de la siguiente manera:

```
kylos@DELL7200:~$ docker container ls -a
CONTAINER ID   IMAGE          COMMAND         CREATED        STATUS
PORTS         NAMES
edc6b2d50ec3   hola-mundo    "/hello"       12 minutes ago Exited (0) 11 minutes ago
               xenodochial_visvesvaraya
kylos@DELL7200:~$
```

Donde, a diferencia de haber corrido el comando anterior, aquí se nos muestra un contenedor, el cual es el que creé, dando a entender que el contenedor no está en ejecución, entonces puedo decir que no hay ningún contenedor ejecutándose.

Actividad 4

El ejercicio de esta primera parte consiste en complicarlo un poco más. Se creará un nuevo contenedor a partir de una imagen ya existente: el contenedor de la distribución [Ubuntu](#). Para ello ejecutar el comando:

```
docker run --interactive --tty ubuntu bash
```

se esta arrancando un contenedor que ejecuta el comando `bash` (intérprete de comandos) desde una imagen de la distribución Ubuntu.

Recordando. **Un contenedor es una *imagen arrancada* y que ejecuta una única aplicación y una imagen es un *contenedor congelado*.** Entonces utilizando la imagen de Ubuntu que se descarga automáticamente cuando no se encuentra localmente al crear el contenedor con `$docker run`.

Una vez que se creo este contenedor (se debe ver que el prompt cambia a algo como `root@CONTAINERID`, donde `CONTAINERID` es el identificador del contenedor). Esto significa que se está dentro del contenedor y se tiene acceso mediante consola. Desde allí se instalará un paquete para la distribución Ubuntu para la aplicación a crear.

```
apt-get update && apt-get install figlet
```

Pregunta: ¿qué hace el paquete de software (figlet)?

Entonces, primero comienzo creando un nuevo contenedor, siendo el de la distribución Ubuntu, de acuerdo al comando compartido por la profesora en la actividad.


```
kylos@DELL7200:~$ docker run --interactive --tty ubuntu bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
0622fac788ed: Pull complete
Digest: sha256:6015f66923d7afbc53558d7ccffd325d43b4e249f41a6e93eef074c9505d2233
Status: Downloaded newer image for ubuntu:latest
root@2229ab4a5d83:/#
```

Después, procedo con la instalación del paquete que se nos solicite que se instale en el contenedor, quedando de la siguiente manera:

```
root@2229ab4a5d83:/# apt-get update && apt-get install figlet
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1318 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [22.1 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1034 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1081 kB]
Get:12 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1354 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [1362 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1380 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [26.7 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [48.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [31.8 kB]
Fetched 29.9 MB in 7s (4153 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  figlet
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 133 kB of archives.
After this operation, 752 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 figlet amd64 2.2.5-3 [133 kB]
Fetched 133 kB in 0s (270 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package figlet.
(Reading database ... 4381 files and directories currently installed.)
Preparing to unpack .../figlet_2.2.5-3_amd64.deb ...
Unpacking figlet (2.2.5-3) ...
Setting up figlet (2.2.5-3) ...
update-alternatives: using /usr/bin/figlet-figlet to provide /usr/bin/figlet (figlet) in auto mode

update-alternatives: warning: skip creation of /usr/share/man/man6/figlet.6.gz because associated file /usr/share/man/man6/figlet-figlet.6.gz (of link group figlet) doesn't exist
root@2229ab4a5d83:/#
```

Lo que hace el paquete “figlet” que se instaló, es crear texto en forma de arte ASCII, por ejemplo, si ejecuto el comando “figlet Hola Mundo”, generará una salida con el formato mencionado anteriormente, como se ve a continuación.


```
root@e813f8dd2d3f:/# figlet Hola Mundo
```

The text "Hola Mundo" is rendered in a large, stylized font made of white lines on a black background. Each letter is composed of vertical and horizontal segments, with some internal gaps, giving it a digital or ASCII-art appearance.

Y se puede hacer con diferentes textos que queramos, como, por ejemplo:

```
root@d8036518d205:/# figlet Ya No Te Pienso
```

The text "Ya No Te Pienso" is rendered in a large, stylized font made of white lines on a black background. The letters are composed of vertical and horizontal segments, with some internal gaps, giving it a digital or ASCII-art appearance.

```
root@e813f8dd2d3f:/# figlet Flaco Donde Estas
```

The text "Flaco Donde Estas" is rendered in a large, stylized font made of white lines on a black background. The letters are composed of vertical and horizontal segments, with some internal gaps, giving it a digital or ASCII-art appearance. The text is split across two lines: "Flaco Donde" on the first line and "Estas" on the second line.

```
root@e813f8dd2d3f:/# figlet :v
```

The text ":v" is rendered in a large, stylized font made of white lines on a black background. The colon is represented by two small circles, and the 'v' is composed of several vertical and diagonal segments, giving it a digital or ASCII-art appearance.

Actividad 5

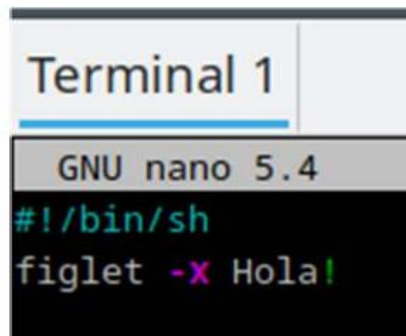
A continuación, se va a crear una sencilla aplicación que será la (única) que será ejecutada en el contenedor.

Todavía dentro se contenedor editar el código de la aplicación con el comando (Cambiar Hola Por sus apellidos):

```
echo -e "#'!'"/bin/sh\nfiglet -x Hola"'!'"' > hola.sh
```

Pregunta: ¿qué función tiene el [comando anterior](#)? ¿Qué son todos esos [caracteres raros](#)? ¿Qué significa el [símbolo !#](#)?

La idea es formar el script siguiente:



A continuación se da permisos de ejecución a este script con el comando:

```
chmod +x hola.sh
```

Se cierra el contenedor (lo apagamos) saliendo de la aplicación que se estaba ejecutando (`bash`). Para salir del intérprete de comandos, simplemente introduce el comando `exit` y pulsa enter.

Ahora, aunque el contenedor se haya parado, lo seguimos teniendo a nuestra disposición para re-arrancarlo cuando queramos. Lo podemos ver en el listado de contenedores con el comando `docker container ls -a`

```
docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
d134db7c1939	ubuntu	"bash"	7 minutes ago	Exited (0) 2 seconds ago
49ea6d5aef73	ubuntu	"bash"	14 minutes ago	Exited (0) 11 minutes ago

El listado anterior nos está diciendo claramente que el contenedor que acabamos de usar es el que concluyó hace apenas unos segundos. Una vez localizado, vamos a crear una imagen a partir del mismo mediante el comando:

```
docker commit CONTAINERID mihola
```

donde CONTAINERID es el identificador del contenedor. Es decir, hemos generado una *instantánea* (foto instantánea) del contenedor y la hemos guardado como una nueva imagen (de la que, a su vez, volveremos a poder crear nuevos contenedores).

`docker commit` es un comando de Docker que se utiliza para crear una nueva imagen a partir de los cambios realizados en un contenedor en ejecución o detenido.

Ejemplo.

```
> docker container ls -a
CONTAINER ID   IMAGE          COMMAND        CREATED        STATUS
15aalf2eabc9   ubuntu        "bash"        12 minutes ago Exited (0) 25 s
b04b6e9f9d71   hello-world   "/hello"      25 minutes ago Exited (0) 25 m

> docker commit 15aalf2eabc9 mihola
sha256:b34eee2d2f98fe7164b832c92fe09ecd67304ccc300a3eec6bc0929396521af5
```

Donde, como se indica en las instrucciones, se empezó primero editando el código de la aplicación con el comando que se nos indica, cambiando el “Hola” por mis apellidos, quedando de la siguiente manera.

```
root@e813f8dd2d3f:/# echo -e "#'!'/bin/sh\\nfiglet -x Silverio Martinez"!'!"
> hola.sh
```

Después, se procedió a listar los archivos para ver si es que “hola.sh” se había creado con éxito, y seguido a esto, se procedió a darle permisos de ejecución con el comando “chmod +x hola.sh”.

```
root@e813f8dd2d3f:/# ls
bin    dev    hola.sh  lib    media  opt    root  sbin  sys  usr
boot  etc    home    lib64  mnt    proc  run   srv   tmp  var
root@e813f8dd2d3f:/# chmod +x hola.sh
```

Después, procedí a salir del intérprete de comandos, y a listar todos los contenedores, esto con el fin de ver que el contenedor recién creado se encontraba parado.

```
kylos@DELL7200:~$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
e813f8dd2d3f	ubuntu	"bash"	25 minutes ago	Exited (0) 9 seconds ago		stupefied_mirzakhani
2229ab4a5d83	ubuntu	"bash"	25 hours ago	Exited (129) 25 hours ago		magical_wescoff
edc6b2d50ec3	hola-mundo	"/hello"	25 hours ago	Exited (0) 25 hours ago		xenodochial_visvesvaraya

Seguido a eso, creé una imagen a partir del mismo contenedor, esto teniendo a la mano el id del contenedor, así como la asignación de un nombre, en este caso “mihola”.

```
kylos@DELL7200:~$ docker commit e813f8dd2d3f mihola
sha256:f65a3a28fd43062b87b060ae151c08bcecaff1d76c135d85623e1893f2149b8e
```

Actividad 6

Listar las imágenes a nuestra disposición, se verá que se encuentra entre ellas:

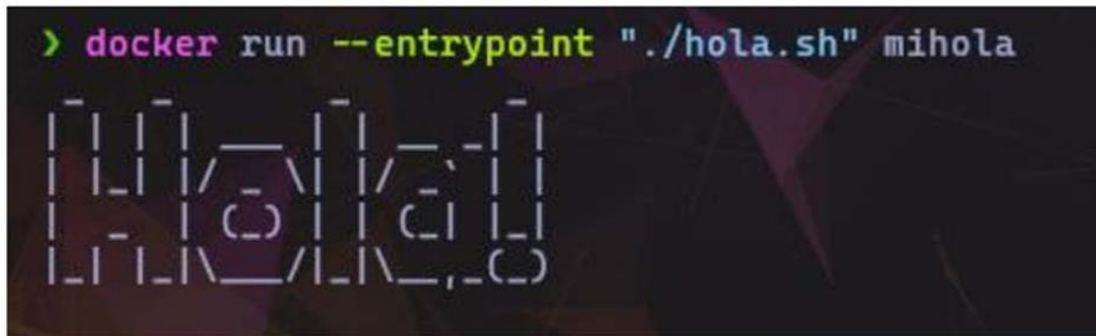
```
docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mihola2	latest	cd16a3e8704e	5 minutes ago	122MB
mihola	latest	9c53553ec8a8	16 minutes ago	122MB
ubuntu	latest	452a96d81c30	3 weeks ago	79.6MB

A continuación, desde la imagen creada (mihola), se creará y arrancará un nuevo contenedor.

```
docker run --entrypoint "/hola.sh" mihola
```

Al hacerlo se observa el resultado que se indicó en el script del contenedor



Por otro lado, Docker le asigna a este contenedor un nombre al azar (modest_lamport en este caso):

```
docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	NAMES
98b61a4dd452	mihola3	"/hola.sh"	2 seconds ago	modest_lamport

Retomando parte de la actividad anterior, listamos todas las imágenes que se han creado, donde aparece “mihola”, el cual fue creado en la actividad anterior.

```
kylos@DELL7200:~$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mihola	latest	f65a3a28fd43	About a minute ago	128MB
ubuntu	latest	a0e45e2ce6e6	2 weeks ago	78.1MB
hola-mundo	latest	aa08f42dc496	6 years ago	1.86kB

Ahora, desde la imagen creada, que es “mihola”, se creará y arrancará un nuevo contenedor, mostrando como salida mis apellidos.

```
kylos@DELL7200:~$ docker run --entrypoint "/hola.sh" mihola
```



Después, creado el contenedor a partir de la imagen, listamos todos los contenedores para ver como aparece el contenedor, que si somos observadores, aparece que fue creado a partir de la imagen “mihola”.

```
kylos@DELL7200:~$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8cb4ac1630a2	mihola	"/hola.sh"	42 seconds ago	Exited (0) 41 seconds ago		eager_meitner
e813f8dd2d3f	ubuntu	"bash"	29 minutes ago	Exited (0) 4 minutes ago		stupefied_mirzakhani
2229ab4a5d83	ubuntu	"bash"	25 hours ago	Exited (129) 25 hours ago		magical_wescoff
edc6b2d50ec3	hola-mundo	"/hello"	26 hours ago	Exited (0) 26 hours ago		xenodochial_visvesvaray

Actividad 7

En este ejemplo se indica la aplicación por defecto del contenedor (con la opción `--entrypoint`). Pero existe otra manera mejor de definir contenedores y la funcionalidad de cada uno, además de otras opciones. Esto se hace mediante un archivo llamado Dockerfile. Los Dockerfiles son los archivos de configuración que contienen instrucciones para la definición de un contenedor.

Crear un archivo con el nombre `HolaMundo.Dockerfile` y colocar las instrucciones

```
FROM ubuntu
RUN apt update && apt install figlet -y
CMD ["/usr/bin/figlet", "Hola desde un Dockerfile"]
```

Ahora se creará una imagen y se lanzará un contenedor con ella.

Crear imagen

```
$docker build -f HolaMundo.Dockerfile .
```

Docker construye la imagen, añadiendo todo el software necesario.

Al final concluirá con un mensaje similar a :

- Successfully built f2bf671ccd6f (idimagen)

Lanzar contenedor

```
$docker run f2bf671ccd6f
```

Para esta actividad, se empezó con la creación del archivo “`HolaMundo.Dockerfile`”.

```
kylos@DELL7200:~$ nano HolaMundo.Dockerfile
```


En dicho archivo se colocan las siguientes líneas de código, con el propósito de “automatizar” los procesos necesarios para crear el contenedor

```
FROM ubuntu
RUN apt update && apt install figlet -y

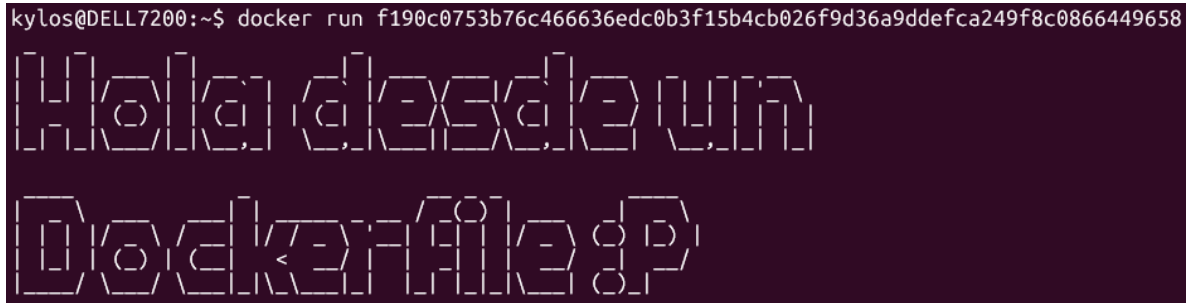
CMD ["/usr/bin/figlet", "Hola desde un Dockerfile :P"]
```

Una vez puesto las líneas de código, y haber guardado al archivo con las líneas de código que se mostraron anteriormente, procedí a crear la imagen a partir del archivo recién creado, mediante el comando “docker build -f <nombredelarchivo>”

```
kylos@DELL7200:~$ docker build -f HolaMundo.Dockerfile .
[+] Building 5.2s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
[+] Building 5.4s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s => #
md64 Packages [48.0 kB]                                           0.0s => #
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s => #
[+] Building 5.5s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s
[+] Building 5.7s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s
[+] Building 5.8s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s
[+] Building 6.0s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
[+] Building 6.1s (4/5)                                docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s
[+] Building 9.1s (6/6) FINISHED                            docker:default
=> [internal] load build definition from HolaMundo.Dockerfile      0.1s
=> == transferring dockerfile: 154B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:latest    0.0s
=> [internal] load .dockerignore                                    0.0s
=> == transferring context: 2B                                       0.0s
=> [1/2] FROM docker.io/library/ubuntu:latest                      0.0s
=> [2/2] RUN apt update && apt install figlet -y                    8.5s
=> exporting to image                                              0.3s
=> == exporting layers                                              0.3s
=> == writing image sha256:97223561cae27785047ffbb762547930fad85b2b3c1114a5d6255f292eb8c7ee 0.0s
kylos@DELL7200:~$
```

Al final, concluye con el id del docker recién creado, donde a partir de ella, lo corremos, para checar que todo haya quedado bien.

```
kylos@DELL7200:~$ docker run f190c0753b76c466636edc0b3f15b4cb026f9d36a9ddefca249f8c0866449658
```



Actividad 8

Investigar como borrar contenedores y después listar los contenedores que están parados y borrarlos

Hay dos formas para borrar contenedores, la primera es para borrar contenedores específicos, para ello, primero debemos de conocer el id del contenedor que deseamos borrar, esto por medio del comando “docker ps -a”.

```
kylos@DELL7200:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1dc19195641c	f190c0753b76	"/usr/bin/figlet 'Ho..."	7 minutes ago	Exited (0) 7 minutes ago		determined_borg
8f50bf7390f7	97223561cae2	"/usr/bin/figlet 'Ho..."	7 minutes ago	Exited (0) 7 minutes ago		interesting_mclaren
c3654481f565	97223561cae2	"/usr/bin/figlet 'Ho..."	7 minutes ago	Exited (0) 7 minutes ago		funny_kalam
ab7bb02f6c5c	97223561cae2	"/usr/bin/figlet 'Ho..."	20 hours ago	Exited (0) 20 hours ago		elegant_nash
8cb4ac1630a2	miholu	"/hola.sh"	20 hours ago	Exited (0) 20 hours ago		eager_meitner
e813f8dd2d3f	ubuntu	"bash"	21 hours ago	Exited (0) 20 hours ago		stupefied_mirzakhani
2229ab4a5d83	ubuntu	"bash"	45 hours ago	Exited (129) 45 hours ago		magical_wescoff
edc6b2d50ec3	hola-mundo	"/hello"	46 hours ago	Exited (0) 46 hours ago		xenodochial_visvesvaraya

En este caso voy a borrar el primero, el que tiene como id “1dc19195641c”. Después, se utiliza el comando “docker rm 1dc19195641c” para eliminarlo definitivamente.

```
kylos@DELL7200:~$ docker rm 1dc19195641c
1dc19195641c
```

Donde nos termina regresando el id del contenedor que acabamos de borrar. Ahora bien, listando los contenedores, después de haber borrado este, ya no aparece.

```
kylos@DELL7200:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8f50bf7390f7	97223561cae2	"/usr/bin/figlet 'Ho..."	9 minutes ago	Exited (0) 9 minutes ago		interesting_mclaren
c3654481f565	97223561cae2	"/usr/bin/figlet 'Ho..."	9 minutes ago	Exited (0) 9 minutes ago		funny_kalam
ab7bb02f6c5c	97223561cae2	"/usr/bin/figlet 'Ho..."	20 hours ago	Exited (0) 20 hours ago		elegant_nash
8cb4ac1630a2	miholu	"/hola.sh"	20 hours ago	Exited (0) 20 hours ago		eager_meitner
e813f8dd2d3f	ubuntu	"bash"	21 hours ago	Exited (0) 20 hours ago		stupefied_mirzakhani
2229ab4a5d83	ubuntu	"bash"	45 hours ago	Exited (129) 45 hours ago		magical_wescoff
edc6b2d50ec3	hola-mundo	"/hello"	46 hours ago	Exited (0) 46 hours ago		xenodochial_visvesvaraya

Ahora bien, la otra forma de borrar contenedores, es borrar todos los que están parados, esto mediante el comando “docker container prune”, pero antes de borrarlos, mostramos los contenedores que están parados, mediante “docker ps -a”.

```
kylos@DELL7200:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
8f50bf7390f7	97223561cae2	"/usr/bin/figlet 'Ho..."	18 minutes ago	Exited (0)	18 minutes ago	interesting_mclaren
c3654481f565	97223561cae2	"/usr/bin/figlet 'Ho..."	18 minutes ago	Exited (0)	18 minutes ago	funny_kalam
ab7bb02f6c5c	97223561cae2	"/usr/bin/figlet 'Ho..."	20 hours ago	Exited (0)	20 hours ago	elegant_nash
8cb4ac1630a2	mihola	"/hola.sh"	20 hours ago	Exited (0)	20 hours ago	eager_meitner
e813f8dd2d3f	ubuntu	"bash"	21 hours ago	Exited (0)	20 hours ago	stupefied_mirzakhani
2229ab4a5d83	ubuntu	"bash"	45 hours ago	Exited (129)	45 hours ago	magical_wescoff
edc6b2d50ec3	hola-mundo	"/hello"	46 hours ago	Exited (0)	46 hours ago	xenodochial_visvesvaraya

Ahora si, se utiliza el comando “docker container prune”

```
kylos@DELL7200:~$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
8f50bf7390f7390f7884d00527f892954f7a69743aaaa01cacc4c19ae6b56edef57ad
c3654481f5658d97f4340dde18276966f892380be684581fedb419d42282de36
ab7bb02f6c5cd0179a26a178371dcecd05e0556397ad1d8077bb88930c98ea25
8cb4ac1630a2824a147193fa21107bb987dbbfcebfec1e43645ad5d556370f4
e813f8dd2d3fee3c348dbb2070cde4f70470f094a88a7efbf3594aa70499ecd7
2229ab4a5d83fdc78f7cc48ba436c08b192a6ed50caf2b46fd918bf86f2efa84
edc6b2d50ec321de5a875d232239f2ba4758e3a0390d4957b9b78457f00e1283

Total reclaimed space: 99.5MB
```

Donde, al momento de ejecutar el comando, se nos pregunta si queremos eliminar todos los contenedores que están parados, aceptamos, y seguido a esto imprimirá el id de los contenedores borrados, y un pequeño mensaje de cuanto espacio liberó el borrar dichos contenedores. Ahora, se vuelve a correr el comando “docker ps -a” para checar si tenemos algún contenedor parado que no haya sido borrado.

```
kylos@DELL7200:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Donde, como se muestra, ha sido un éxito el borrado de todos los contenedores que se encontraban parados.